

Module : Algorithmes et structures de données I

TD 4 : Les tableaux et les sous-programmes

Exercice 1 : Chargement d'un tableau par des entiers.

Écrire un algorithme permettant de :

- Saisir un entier positif N1.
- Compter puis d'afficher le nombre de chiffres 1 qu'on peut trouver entre 0 et N1.
- Mémoriser entre-temps chaque nombre qui contient le chiffre 1 dans un tableau T puis afficher le contenu de ce tableau.

Exemple :

Donnée :

N1 = 56

Résultat :

Nombre d'occurrences de chiffres 1 = 16

N = 15

T

1	10	11	12	13	14	15	16	17	18	19	21	31	41	51				
---	----	----	----	----	----	----	----	----	----	----	----	----	----	----	--	--	--	--

Exercice 2 : Traitement des tableaux de caractères.

Écrire les sous-programmes et l'algorithme principal permettant de :

- Saisir un tableau T par les lettres majuscules dans l'ordre croissant.
- Saisir 10 lettres majuscules toutes différentes et les stocker dans un tableau T1.
- Mémoriser les lettres qui existent dans T et n'existent pas dans T1 dans un tableau T2.

Exercice 3 : Inversion de tableaux.

Écrire les sous-programmes et l'algorithme principal permettant de :

- Charger un tableau T1 par n1 entier avec $3 \leq n1 \leq 20$.
- Charger un tableau T2 par n2 entier avec $3 \leq n2 \leq 20$.
- Inverser T1 et T2.
- Vérifier si T1 est inclus dans T2 (c.à.d. tous les éléments de T1 existent dans T2).

Exemple :

Donnée : T1

2	8	4
---	---	---

Résultat : Le tableau T1 est inclus dans T2

T2

4	5	8	9	2
---	---	---	---	---

Exercice 4 : Suppression des redondances d'un tableau.

Écrire les sous-programmes et l'algorithme principal permettant de :

- Remplir un vecteur T par des entiers strictement positifs.
- Éliminer toutes les redondances de T en mettant à 0 chaque valeur dupliquée du tableau.
- Arranger le tableau de façon à avoir les valeurs non nulles au début du tableau (en respectant le même ordre d'apparition) suivies par les valeurs nulles si elles existent. N'utiliser pas un tableau supplémentaire.
- Afficher les éléments non nuls du tableau.

Exercice 5 : Suppression d'un sous tableau.

Écrire les sous-programmes et l'algorithme principal permettant de :

- Remplir deux tableaux A et B avec des nombres positifs.
- Éliminer la troisième occurrence du tableau B dans le tableau A.

Exemple :

Donnée : B

2	8	4
---	---	---

A

1	2	2	8	4	4	2	8	2	8	4	9	2	8	4	0	2
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Résultat :

A

1	2	2	8	4	4	2	8	2	8	4	9	0	2	4	0	2
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Exercice 6 : Matrice diagonale.

On appelle *matrice diagonale* une matrice carrée dont les termes situés hors de la diagonale principale sont tous nuls. Ecrire un sous-programme qui permet de vérifier si une matrice M de taille n est diagonale.

Exemple : la matrice M suivante est diagonale (pour i, avec $1 \leq i \leq n$ et j tel que, $i \neq j$, $M[i,j]=0$)

4	0	0	0
0	0	0	0
0	0	2	0
0	0	0	3

Exercice 7 : Matrice unité.

Une *matrice diagonale* dont les éléments de la diagonale principale sont tous égaux à 1 est appelée *matrice unité*. Ecrire un sous-programme qui permet de vérifier si une matrice M de taille n est unité.

Exemple : la matrice M suivante est unité (pour i, avec $1 \leq i \leq n$ et j tel que, $i \neq j$, $M[i,j]=0$ et $M[i,i]=1$)

1	0	0	0
0	1	0	0
0	0	1	0
0	0	0	1

Exercice 8 : Matrice symétrique.

Écrire les sous-programmes et l'algorithme principal permettant de saisir une matrice carrée M de taille n et de déterminer si elle est symétrique.

Exemple : la matrice M suivante est symétrique ($M[i,j]=M[j,i] \forall i \text{ et } j$)

4	5	7	9
5	1	2	8
7	2	2	6
9	8	6	3

Exercice 9 : Matrice parfaite.

Ecrire un sous-programme qui permet de déterminer si une matrice est parfaite. Une matrice est parfaite si la somme de chaque ligne ou de chaque colonne donne la même valeur.

Exercice 10 : recherche dichotomique d'une valeur dans les lignes d'une matrice.

Ecrire les sous-programmes et l'algorithme principal permettant de remplir une matrice avec des nombres positifs, en s'assurant que chaque ligne est croissante puis d'effectuer une recherche dichotomique d'une valeur donnée par l'utilisateur dans les différentes lignes de la matrice. On arrête la recherche dès que la valeur est trouvée.

Exercice 11 : Matrice triangulaire.

Ecrire un sous-programme qui vérifie si une matrice est triangulaire. Une matrice est dite triangulaire si la somme des quatre triangles est la même.

Exemple : la matrice suivante est triangulaire ($S1=S2=S3=S4(=14)$)

	S1			
	2	5	4	
	3	2	3	5
S3	5	3	1	2
	3	7	4	2
	4	2	4	5
		S2		S4